

Few-shot Adaptation of SmolVLA on LIBERO

Mikhail Orekhov

T-Lab Test Assignment, T-Bank

[GitHub Repository](#)

August 2026

1 Task Setup

The goal of this work is to shift the SmolVLA adaptation cost curve on new LIBERO tasks: to increase success rate at a fixed number of target demonstrations (0, 5, 10, and 25). The starting model is `lerobotsmolvla_base`. It consists of a pretrained VLM and an action expert that predicts a continuous action chunk with conditional flow matching [1]. In the experiments, the VLM is frozen; the action expert and the action/state projectors enabled in the configuration are trained. The model predicts 50 actions, but the environment executes one action before the next observation.

For seen pretraining, I used a fixed stratified subset of 40 tasks from canonical `libero_90`: 20 kitchen, 13 living room, and 7 study tasks. All 50 demonstrations were used for each task; the seen pool therefore contains 2 000 episodes. The subset was selected before target results were obtained, contains no `libero_goal` demonstrations, and covers all three domains. This constraint preserved the compute budget for two seen seeds, the full baseline grid, and ablations. The source HDF5 demonstrations are read on the fly; the target-demo order is fixed by the numeric IDs `demo_0`, ..., `demo_24`.

The target tasks are the first three `libero_goal` tasks by benchmark ID: *open the middle drawer of the cabinet*, *put the bowl on the stove*, and *put the wine bottle on top of the cabinet*. All final points use two train seeds (0, 1), 20 online rollouts per task-seed, and identical initial-state seeds 10000, ..., 10019. The metric in the tables is success rate, averaged over two train seeds and three target tasks.

2 Seen Pretraining, Zero-Shot, and Baseline

Seen pretraining lasted 10 000 optimizer updates with an effective batch size of 32. AdamW ($\text{lr} = 10^{-4}$, $\beta = (0.9, 0.95)$), a linear warmup of 1 000 updates, and cosine decay were used. For each target point, the baseline is fine-tuned for 2 000 updates with the same optimizer, trainable modules, preprocessing, and action interface; the warmup is 250 updates. Thus, the baseline is a naïve fine-tune of the seen checkpoint on the first 5/10/25 demonstrations of one target task, without replay, additional losses, or augmentations.

Zero-shot success is 0% on all three target tasks for both seen seeds. In the language-control check on task 0, the instruction was replaced with the instruction from task 1; the result is also 0%. This does not imply that the model ignores language; it only shows that in this zero-shot setting, replacing the instruction does not change the observed success.

Table 1: Fixed baseline cost curve. Task values are averages over two train seeds; for budgets 5/10/25, the parentheses report the 95% Wilson interval over 40 rollout outcomes.

Budget (demos)	Task 0	Task 1	Task 2	Mean
0	0.0	0.0	0.0	0.0
5	27.5 [16.1, 42.8]	100.0 [91.2, 100.0]	37.5 [24.2, 53.0]	55.0
10	37.5 [24.2, 53.0]	90.0 [76.9, 96.0]	67.5 [52.0, 79.9]	65.0
25	45.0 [30.7, 60.2]	95.0 [83.5, 98.6]	70.0 [54.6, 81.9]	70.0

Task 1 is already close to the ceiling with five demonstrations; therefore, method improvement should be interpreted through the mean cost curve and tasks 0 and 2.

3 Predictions

Replay buffer. In the replay method, target and seen demonstrations are mixed at a 3 : 1 ratio while keeping the effective batch size at 32. Prediction: replay will reduce forgetting of the seen dataset and improve adaptation on target tasks. The largest effect is expected with a budget of five demonstrations, especially on tasks 0 and 2; at larger budgets, a smaller but positive gain is expected. This hypothesis is motivated by multi-task adaptation while retaining a diverse robot prior [2, 3].

L2-SP. L2-SP does not use seen demonstrations during target fine-tuning. Before adaptation, a copy θ_{seen} of all trainable parameters is saved, and the loss is defined as

$$\mathcal{L} = \mathcal{L}_{\text{flow}} + \lambda \sum_{p \in \mathcal{P}} \|p - p_{\text{seen}}\|_2^2,$$

where $\mathcal{P}$ is the set of trainable parameters. In the experiments, $\lambda = 10^{-2}$. Prediction: L2-SP will have an effect similar to replay, but without reducing the number of target minibatches. As with replay, the largest gain is expected with a budget of five demonstrations; smaller but positive effects are also expected at budgets 10 and 25.

LoRA. LoRA is a parameter-efficient adaptation method motivated by the goal of preserving pretrained representations when few target demonstrations are available [4]. Prediction: restricting the number of trainable degrees of freedom will reduce overfitting and yield the largest gain in the low-budget regime, especially with five demonstrations.

4 Adaptation Results

Replay buffer. Screening on task 0, budget 5, seeds 0 and 1, and task 2, budget 5, seed 0 did not show a consistent improvement over the baseline. Therefore, replay was not run on the full grid. This is a preliminary observation, not evidence of the absence of an effect.

L2-SP. The full L2-SP grid uses the same two seen checkpoints, the first 5/10/25 target demonstrations, 2000 target updates, and fixed-RNG evaluation as the baseline. Each cell below is an average over two train seeds and 20 rollouts per seed (40 rollouts per task and budget in total). The arrow denotes the transition from baseline to L2-SP; the parentheses show the difference in percentage points.

Table 2: Comparison of baseline and L2-SP success rate (%).

Budget	Task 0	Task 1	Task 2	Mean over tasks
5	27.5 → 42.5 (+15.0)	100.0 → 95.0 (−5.0)	37.5 → 35.0 (−2.5)	55.0 → 57.5 (+2.5)
10	37.5 → 67.5 (+30.0)	90.0 → 92.5 (+2.5)	67.5 → 72.5 (+5.0)	65.0 → 77.5 (+12.5)
25	45.0 → 55.0 (+10.0)	95.0 → 92.5 (−2.5)	70.0 → 67.5 (−2.5)	70.0 → 71.7 (+1.7)

L2-SP did not produce a consistent improvement at all budgets. At budget 10, the mean gain is +12.5 p.p. and is primarily driven by task 0 (+30.0 p.p.). At budgets 5 and 25, the mean changes are only +2.5 and +1.7 p.p.; they are accompanied by changes in opposite directions across tasks. For these two budgets, the 95% Wilson intervals for task-level L2-SP and baseline estimates overlap. Thus, the result does not support the initial prediction of a strong and consistent L2-SP advantage specifically in the low-budget regime; in particular, interval overlap should not be interpreted as a formal test of the absence of an effect.

LoRA. To test the overfitting hypothesis in the most data-scarce regime, LoRA was run on the full grid of budgets 5/10/25. LoRA adapts the action-expert attention q/v projections and action/state projections with rank 8, $\alpha = 16$, and dropout 0.05.

Table 3: Comparison of baseline and LoRA success rate (%). Each cell is an average over two train seeds and 20 rollouts per seed; the parentheses show the difference in percentage points.

Budget	Task 0	Task 1	Task 2	Mean over tasks
5	27.5 → 65.0 (+37.5)	100.0 → 92.5 (−7.5)	37.5 → 65.0 (+27.5)	55.0 → 74.2 (+19.2)
10	37.5 → 60.0 (+22.5)	90.0 → 92.5 (+2.5)	67.5 → 57.5 (−10.0)	65.0 → 70.0 (+5.0)
25	45.0 → 65.0 (+20.0)	95.0 → 95.0 (0.0)	70.0 → 62.5 (−7.5)	70.0 → 74.2 (+4.2)

The largest observed LoRA gain is +19.2 p.p. at budget 5 and is driven by tasks 0 and 2. At budgets 10 and 25, the mean shifts decrease to +5.0 and +4.2 p.p.: LoRA consistently improves task 0 but underperforms the baseline on task 2. Task 1 is close to the ceiling and leaves no room for a systematic gain. Thus, the result supports the prediction that LoRA is most useful in the low-budget regime, but does not establish a universal advantage for every task and budget: only two independent training seeds remain for each method.

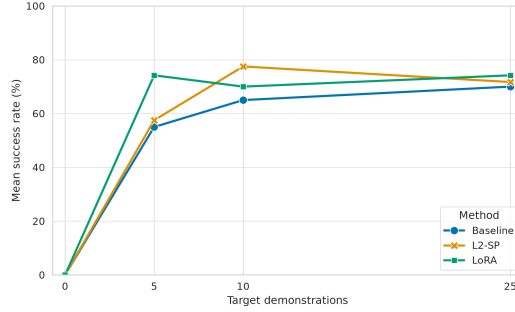
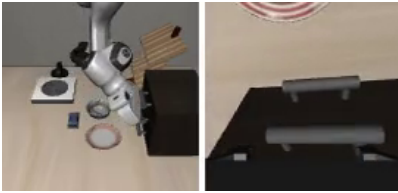


Figure 1: Cost curves of adaptation methods: mean success rate over the three LIBERO target tasks.

5 Failure Analysis



Baseline, task 0, budget 5



L2-SP, task 0, budget 5



Baseline, task 2, budget 5

Baseline, task 0, budget 5, seed 0: handle contact without opening. The task is to open the middle drawer of the cabinet. The end effector approaches the handle of the middle drawer very closely, but does not establish a stable grasp and pulling motion; the drawer remains closed. My hypothesis is that the model correctly localizes the target but cannot reliably transition from visual contact to physical interaction. An alternative explanation is a small error in estimating the 6D pose of the handle: visually, the gripper is close, but it does not actually reach a grasp configuration.

To distinguish these explanations, I would record on the same initial states the end-effector pose, gripper command and aperture, distance to the handle, and drawer joint position. If the gripper receives a closing command near the handle but the drawer does not move, this would support the contact/pulling hypothesis. If the minimum distance and orientation differ substantially from the expert trajectory, geometric localization of the target is the more likely cause.

L2-SP, task 0, budget 5, seed 0: selecting the upper rather than the middle drawer. Despite the instruction “open the middle drawer of the cabinet”, the robot moves toward the upper-drawer region and does not open the middle drawer. L2-SP may have retained a strong seen prior for upper-drawer manipulation and insufficiently adapted the spatial grounding of the words *middle* and *top*. An alternative explanation is an error in visually localizing the drawers because of viewpoint or partial occlusion.

A discriminating experiment would provide two instructions on the same set of initial states that differ only in the word *middle/top*, then compare the early end-effector trajectories and the displacement of both drawer joints with the baseline. If L2-SP systematically moves toward the upper drawer for both instructions while the baseline changes targets, this would support the hypothesis of weak language–action binding. If drawer selection changes with viewpoint but not text, an error in visual-spatial perception is more likely.

Baseline, task 2, budget 5, seed 0: failed bottle grasp. In the task “put the wine bottle on top of the cabinet”, the gripper approaches the bottle, but contact tips it over before the gripper closes. The robot then continues the lifting and placement trajectory and opens the gripper at the top of the cabinet, despite not holding the bottle. The first hypothesis is imprecise grasp geometry: the gripper position or orientation, or the direction of movement immediately before grasping, causes lateral contact and tips over the bottle. The second hypothesis is that after a failed grasp,

the policy insufficiently accounts for the changed object state and continues toward the placement location instead of attempting another grasp.

To test the first hypothesis, I would compare bottle and gripper poses and the motion direction immediately before contact with the expert trajectory. A systematic difference in gripper position or orientation before tipping would support the grasp-error hypothesis. To test the second hypothesis, I would separately analyze the actions after the bottle falls. If, after receiving a new observation in which the bottle is already outside the gripper, the policy continues lifting and moving toward the placement location without attempting to return to the object, this would support the hypothesis of insufficient behavioral correction after a failed grasp.

6 Bonus Task

The bonus task was not completed: the available compute budget and time were allocated to two train seeds, the full baseline grid, and testing adaptation methods in the main part of the assignment.

7 Summary and Discussion

Predictions versus Results. The prediction for L2-SP was not confirmed in its original form. I expected the largest gain with five demonstrations, but the mean gain was only +2.5 p.p. The most pronounced effect was obtained at budget 10 on task 0: 37.5% $\rightarrow$ 67.5% (+30.0 p.p.; both train seeds improve), while the mean gain across the three tasks was +12.5 p.p. At budget 25, the mean shift of +1.7 p.p. is within the available uncertainty.

The LoRA prediction is confirmed on the full grid, but primarily in the low-budget regime. With five demonstrations, mean success increases from 55.0% to 74.2%. On task 0, LoRA with five demonstrations reaches 65.0%, exceeding the baseline even with 25 demonstrations (45.0%), while on task 2, the result of 65.0% is close to the baseline with ten demonstrations (67.5%). At budgets 10 and 25, LoRA retains a mean advantage of +5.0 and +4.2 p.p., respectively, but it is smaller and accompanies a decline on task 2. Thus, a convincing leftward shift of the cost curve is observed mainly with five demonstrations and should not be interpreted as an identical effect across all tasks.

Idea Graveyard.

Seen Replay. Replay was rejected after limited screening did not show a consistent improvement on two difficult target tasks. Given the LoRA signal, the available compute was allocated to a more informative evaluation of low-rank adaptation.

Action Head Replacement. Replacing the action head was not included in the main grid: SmolVLA already uses a native continuous flow-matching action expert. Testing another head would require a separate baseline and a different objective, substantially expanding the compute budget; the comparison would therefore no longer isolate few-shot adaptation.

FiberTune. FiberTune regularizes intermediate visual tokens by separating action-predictive and residual representation directions. For SmolVLA, the method would require access to intermediate tokens, teacher/layer selection, and a separate ablation with the VLM unfrozen, so it was excluded before implementation [5].

Unexpected Observation. The naïve baseline reaches 100% on task 1 already at budget 5 for both train seeds, whereas tasks 0 and 2 retain substantial room for improvement. A likely explanation is that task 1 requires a less precise manipulation sequence: placing a bowl on the stove tolerates a larger final-position error, whereas opening a drawer requires precise contact with the handle, and placing a bottle on the cabinet requires a stable grasp and a longer transfer sequence.

References

1. M. Shukor et al. *SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics*. arXiv:2506.01844, 2025. <https://arxiv.org/abs/2506.01844>
2. Octo Model Team. *Octo: An Open-Source Generalist Robot Policy*. arXiv:2405.12213, 2024. <https://arxiv.org/abs/2405.12213>
3. K. Black et al. π_0 : *A Vision-Language-Action Flow Model for General Robot Control*. arXiv:2410.24164, 2026. <https://arxiv.org/abs/2410.24164>
4. Y. Jiang et al. *VLA-GSE: Boosting Parameter-Efficient Fine-Tuning in VLA with Generalized and Specialized Experts*. arXiv:2605.06175, 2026. <https://arxiv.org/abs/2605.06175>
5. H. Lin et al. *FiberTune: Preserving Action-Fiber Visual Residuals in Vision-Language-Action Fine-Tuning*. arXiv:2606.08653, 2026. <https://arxiv.org/abs/2606.08653>
6. M. J. Kim et al. *OpenVLA: An Open-Source Vision-Language-Action Model*. arXiv:2406.09246, 2024. <https://arxiv.org/abs/2406.09246>
7. M. J. Kim, C. Finn, and P. Liang. *Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success*. arXiv:2502.19645, 2025. <https://arxiv.org/abs/2502.19645>
8. X. Guo et al. *PriorVLA: Prior-Preserving Adaptation for Vision-Language-Action Models*. arXiv:2605.10925, 2026. <https://arxiv.org/abs/2605.10925>
9. C. Xu, E. Yu, J. Xuan, and J. Lu. *VGAS: Value-Guided Action-Chunk Selection for Few-Shot Vision-Language-Action Adaptation*. arXiv:2602.07399, 2026. <https://arxiv.org/abs/2602.07399>
10. D. M. Nguyen et al. *FOCA: Future-Oriented Conditioning for Data-Efficient Vision-Language-Action Adaptation*. arXiv:2606.20867, 2026. <https://arxiv.org/abs/2606.20867>